

Django 教學 10

帳號登入與Session

第 16 章

從最小範例到專案實作與驗收

本份學習路線

先完成 [09_表單與資料驗證](#)。

- **第 16 章：身份驗證與帳號流程**

每章依序：概念、最小範例、語法、專案對照、實作與驗收。

[全課目錄](#) · [實作手冊對照](#)

第 16 章

身份驗證與帳號流程

完成註冊、登入、POST 登出，說明 session 與 request.user 的關係。

本章的操作環境與成果

先用 LearnBoard 理解表單與擁有權，再對照 LearnMart；兩個專案分別使用自己的資料庫。

完成成果： 完成註冊、登入、POST 登出，說明 session 與 request.user 的關係。

完整範例可依步驟操作；標示「節錄／重排」的程式用來閱讀，不當作整檔覆蓋。
進階頁可回查，但所有基本驗收需完成。

16-1 Authentication 與 Authorization

- **Authentication (身份驗證)**：你是誰？是否已登入？
- **Authorization (授權)**：這個人能做什麼？

已登入不等於：

- 一定是賣家
- 可以編輯任意商品
- 可以查看任意訂單

後面會把授權拆成「登入、角色、物件關係」三層。

16-2 自訂 User 必須很早決定

目前 LearnMart | 節錄／重排 | `config/settings.py`

```
AUTH_USER_MODEL = "marketplace.User"
```

新專案應在第一次 `migrate` 前做這個選擇，而且 custom User model 應出現在其 app 的 `0001_initial` migration；Django 的 swappable dependency 與 migration graph 會依賴這個早期位置。

之後更換涉及外鍵、migration 與資料搬移，並非簡單改一行設定。第 2 章已介紹初始化決策；現在才深入帳號流程。

16-3 三種 User 參照方式不要混為一談

1. 同一個 app 內目前程式可直接參照本地 `User` class。
2. 可重用 app 的 model relation 通常用 `settings.AUTH_USER_MODEL`。
3. 執行時要取得目前 User class，使用 `get_user_model()`。

```
from django.contrib.auth import get_user_model
User = get_user_model()
```

不要說目前每一個 relation 都「實際寫成設定字串」；LearnMart models 目前直接用本地 `User`。

16-4 LearnMart User 繼承 AbstractUser

目前 LearnMart | 節錄／重排 | marketplace/models.py::User

```
class User(AbstractUser):
    class Role(models.TextChoices):
        BUYER = "buyer", "買家"
        SELLER = "seller", "賣家"

    role = models.CharField(
        "身份", max_length=10,
        choices=Role.choices, default=Role.BUYER,
    )
```

AbstractUser 保留 Django 既有帳號、密碼與權限欄位，再加專案欄位。

16-5 Role 是商業角色，不是 Django admin 權限

```
@property
def is_seller(self):
    return self.role == self.Role.SELLER
```

- `role=SELLER`：LearnMart 商城中的賣家
- `is_staff=True`：能否進 Django admin
- `is_superuser=True`：Django 權限檢查通常全部通過

這些概念用途不同，不要用 `is_staff` 代替商業角色。

16-6 教學版的 seller 註冊限制

目前設計取舍

`RegistrationForm` 把 `role` 放在 fields，因此任何註冊者都能自行選「賣家」。

這方便課堂練習，但正式商城通常需要：

- 審核或邀請流程
- 商家資料驗證
- 管理員核准角色

教材必須說清楚「目前能做」不等於「正式環境應這樣做」。

16-7 密碼不是加密後可還原

Django 儲存 salted password hash：

```
algorithm$iterations$salt$hash
```

登入時，Django 用同一演算法計算候選密碼，再比較結果。

- hash 是單向驗證，不是把密碼解密
- salt 不必保密，用來降低相同密碼產生相同結果
- 密碼強度仍需 validators；hash 不能把弱密碼變強

16-8 建立 User 的正確方法

```
User.objects.create_user(  
    username="amy",  
    password="safe-pass-123",  
)
```

或對既有 instance：

```
user.set_password("safe-pass-123")  
user.save(update_fields=["password"])
```

錯誤

`User.objects.create(password="...")` 會把字串當一般欄位值，不會正確 hash。

16-9 `create_user()` 不等於跑過所有 password validators

`create_user()` / `set_password()` 會 hash 密碼，但程式直接呼叫時不會自動替你執行所有表單式密碼強度驗證。

`UserCreationForm` 會：

- 要求兩次密碼一致
- 使用設定中的 password validators
- 正確呼叫 `set_password()`

因此互動式註冊流程適合從它擴充。

16-10 LearnMart RegistrationForm

目前 LearnMart | 節錄／重排 | marketplace/forms.py

```
class RegistrationForm(BootstrapFormMixin, UserCreationForm):  
    email = forms.EmailField(label="電子郵件", required=True)  
  
    class Meta:  
        model = User  
        fields = ("username", "email", "role",  
                  "password1", "password2")
```

email 在這個 form 必填，但目前 model/database 並沒有設定 email unique。

16-11 註冊 View 的完整結果

目前 LearnMart | 節錄／重排 | marketplace/views.py::register

```
if request.method == "POST" and form.is_valid():  
    user = form.save()  
    login(request, user)  
    messages.success(request, "註冊成功，歡迎加入學購！")  
    return redirect("marketplace:home")
```

成功後同時：建立 hashed-password user、建立登入狀態、加入 message、redirect。

16-12 Session：跨 request 記住登入狀態

簡化流程：

登入成功

- Server 建立 session data
- Browser 保存 session id cookie
- 下一次 request 帶 cookie
- Middleware 找回 User

cookie 不應放使用者密碼；它通常只攜帶 session identifier。

16-13 為什麼 `request.user` 存在？

目前 LearnMart | 節錄／重排 | `config/settings.py`

```
MIDDLEWARE = [  
    "django.contrib.sessions.middleware.SessionMiddleware",  
    ...,  
    "django.contrib.auth.middleware.AuthenticationMiddleware",  
]
```

SessionMiddleware 先準備 session；AuthenticationMiddleware 再依 session 將 `request.user` 設為 User 或 AnonymousUser。

16-14 request.user 的兩種狀態

```
if request.user.is_authenticated:  
    ...
```

- 未登入：AnonymousUser
- 已登入：目前 User instance

is_authenticated 是 property，不是 is_authenticated()。

Template 中的 user 來自 auth context processor；View 中使用 request.user。

16-15 LoginView：使用成熟的內建流程

目前 LearnMart | 節錄／重排 | `config/urls.py`

```
path(
    "accounts/login/",
    auth_views.LoginView.as_view(
        template_name="registration/login.html"
    ),
    name="login",
)
```

`.as_view()` 會把 class-based view 轉成 URLconf 可呼叫的 callable；可回查第 13 章的查詢型 CBV 流程。

16-16 `next`：登入後回到原本目的地

匿名使用者造訪 `/cart/`：

```
/accounts/login/?next=/cart/
```

登入表單保留：

```
{% if next %}  
  <input type="hidden" name="next" value="{{ next }}">  
{% endif %}
```

成功後 Django 驗證目標 URL，再導回原頁。

16-17 Logout 應使用 POST

目前 LearnMart | 節錄／重排 | [templates/base.html](#)

```
<form method="post" action="{% url 'logout' %}">
  {% csrf_token %}
  <button type="submit">登出</button>
</form>
```

登出會改變 session 狀態。使用 POST + CSRF 可避免第三方圖片或連結讓使用者在不知情時被登出。

16-18 Login redirects 在 settings 定義

```
LOGIN_URL = "login"  
LOGIN_REDIRECT_URL = "marketplace:home"  
LOGOUT_REDIRECT_URL = "marketplace:home"
```

- `LOGIN_URL` : 未登入保護頁要去哪裡
- `LOGIN_REDIRECT_URL` : 無 `next` 時登入後去哪裡
- `LOGOUT_REDIRECT_URL` : 登出後去哪裡

使用 named URL 比硬編碼 path 更容易維護。

16-19 302、403、404 的身份／權限語意

狀況	常見結果
匿名者造訪需登入頁	302 到 login，附 <code>next</code>
已登入但角色不符	403 Forbidden
物件不屬於目前使用者	404 Not Found

用 404 隱藏「別人的物件是否存在」，是物件層權限的常見做法。

16-20 概念檢核

1. Authentication 與 authorization 有何不同？
2. 為何 custom user 要在初始 migration 前決定？
3. session cookie 是否包含密碼？
4. `create_user()` 與 `UserCreationForm` 各保證什麼？
5. 為何 logout 應使用 POST？

答案見配套實作手冊原第 2 章

16-21 LearnMart 實作

任務：追蹤「匿名進購物車 → 登入 → 回購物車 → POST 登出」完整流程。

驗收重點：

- login URL 含正確 `next`
- 登入後 `request.user` 改變
- session 跨 request 保留
- logout GET 不作為操作入口，POST 含 CSRF

LearnMart 步驟與觀察表；LearnBoard 對應練習

第 16 章實作與離堂檢核

任務： 完成註冊、登入、POST 登出，說明 session 與 request.user 的關係。

1. 展示操作結果或測試紀錄，指出對應檔案與資料。
2. 解釋一個輸入如何得到結果，以及規則在哪一層檢查。
3. 改變一個條件或製造一次失敗，記錄觀察與修正。

配套練習： [LearnBoard 02 原第 2 章](#)；[LearnMart 02 原第 2 章](#)。手冊保留原章號，對照表列出本課位置。

本份完成與後續

下一份：[11_CRUD與物件權限](#)。

- 保留本份操作紀錄，確認使用正確的專案與資料庫。
- 章節與實作對應可由 [全課目錄](#) 回查。